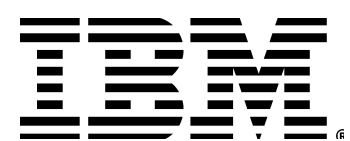


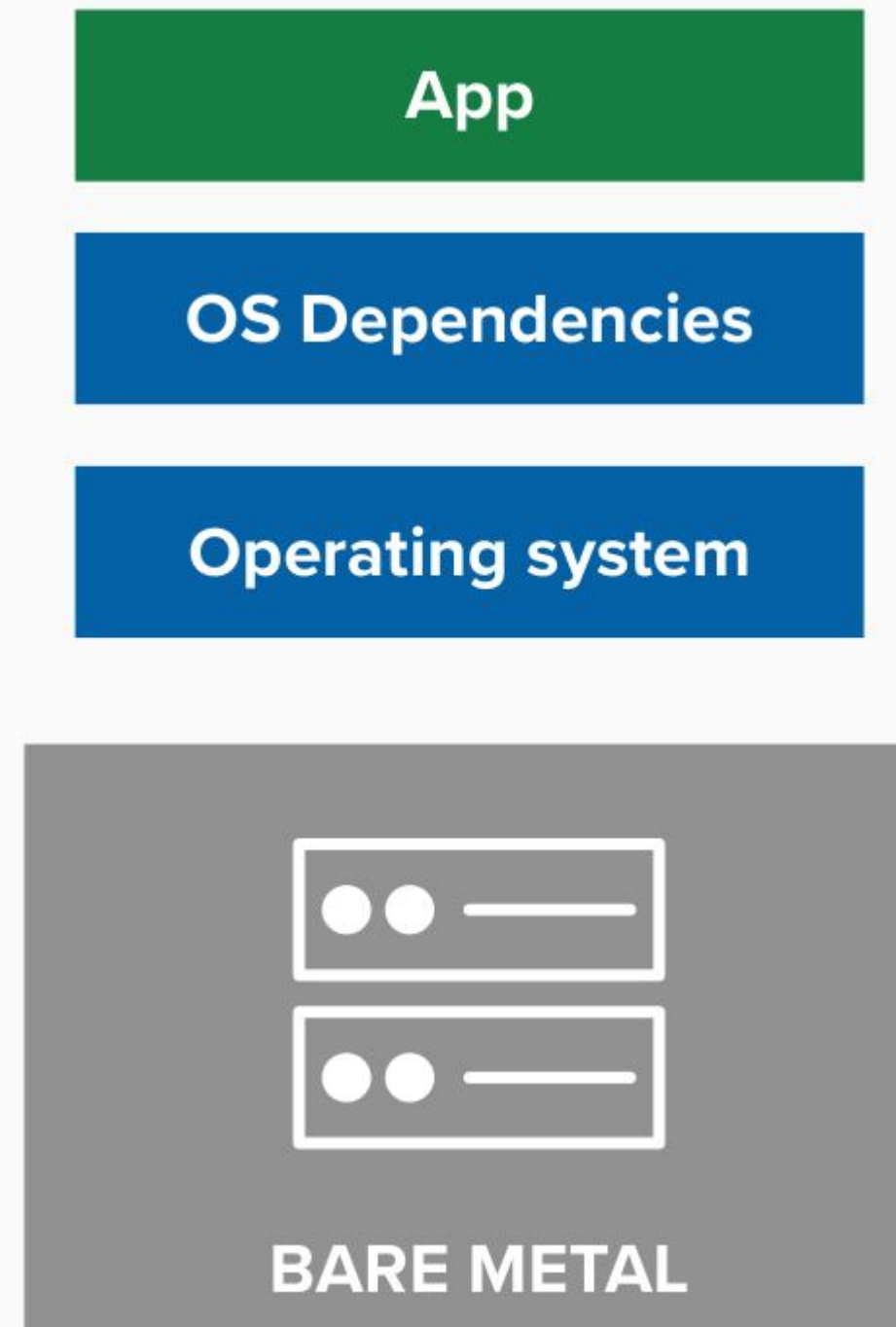
Containers & OpenShift Workshop

Alexander Mamani - IBM
Kate Queiroz - Red Hat
Christian Zaccaria - Red Hat
Winnie Imafidon - IBM

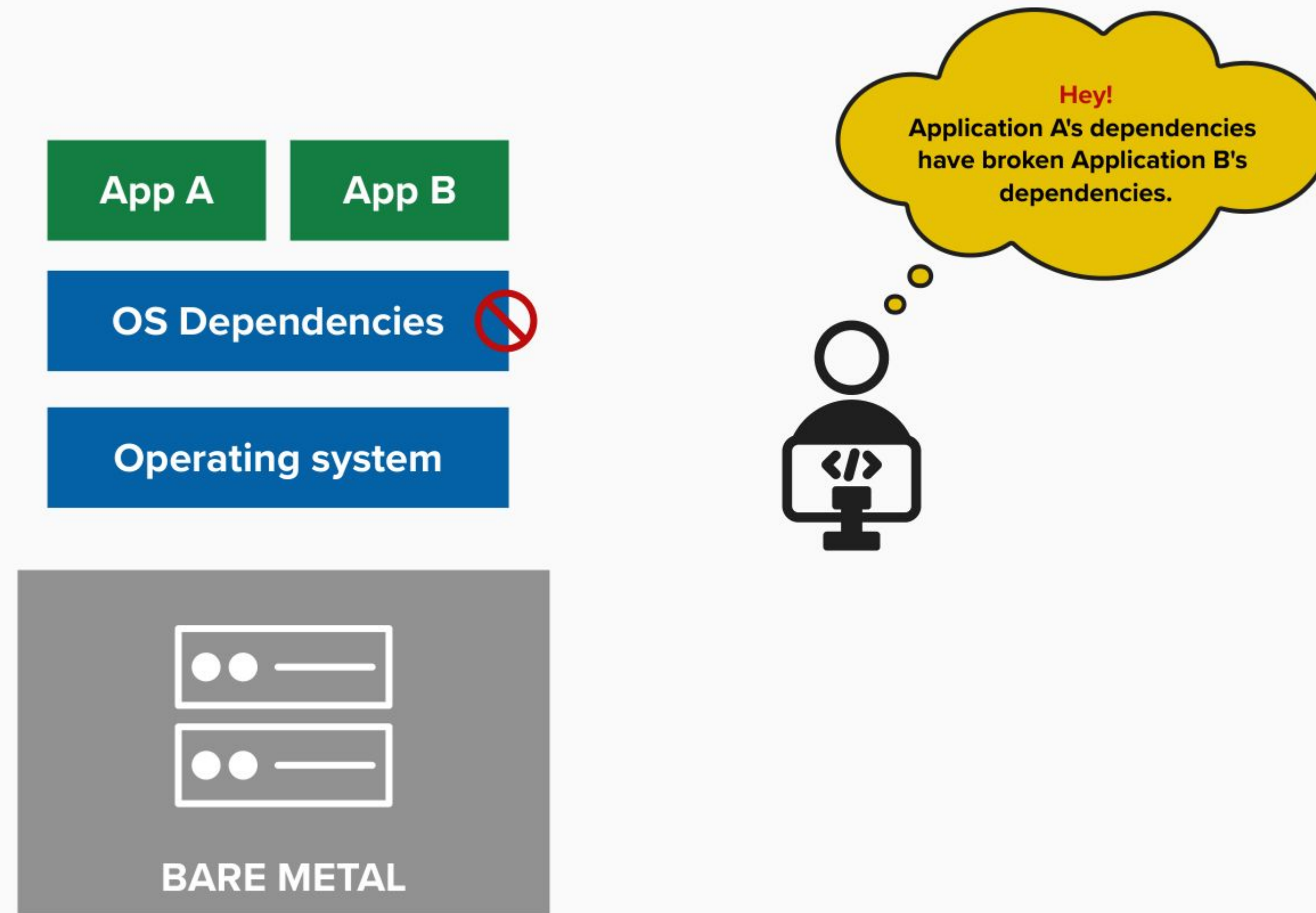


A LITTLE BIT ABOUT VIRTUALIZATION

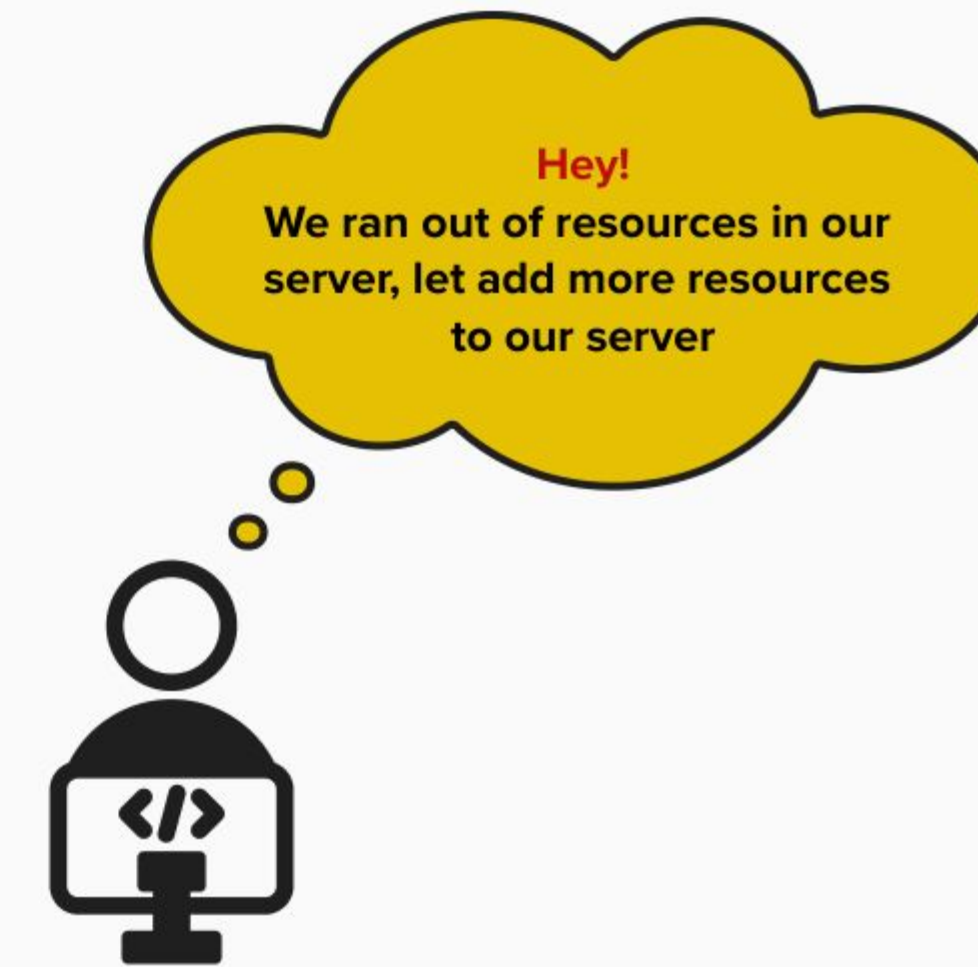
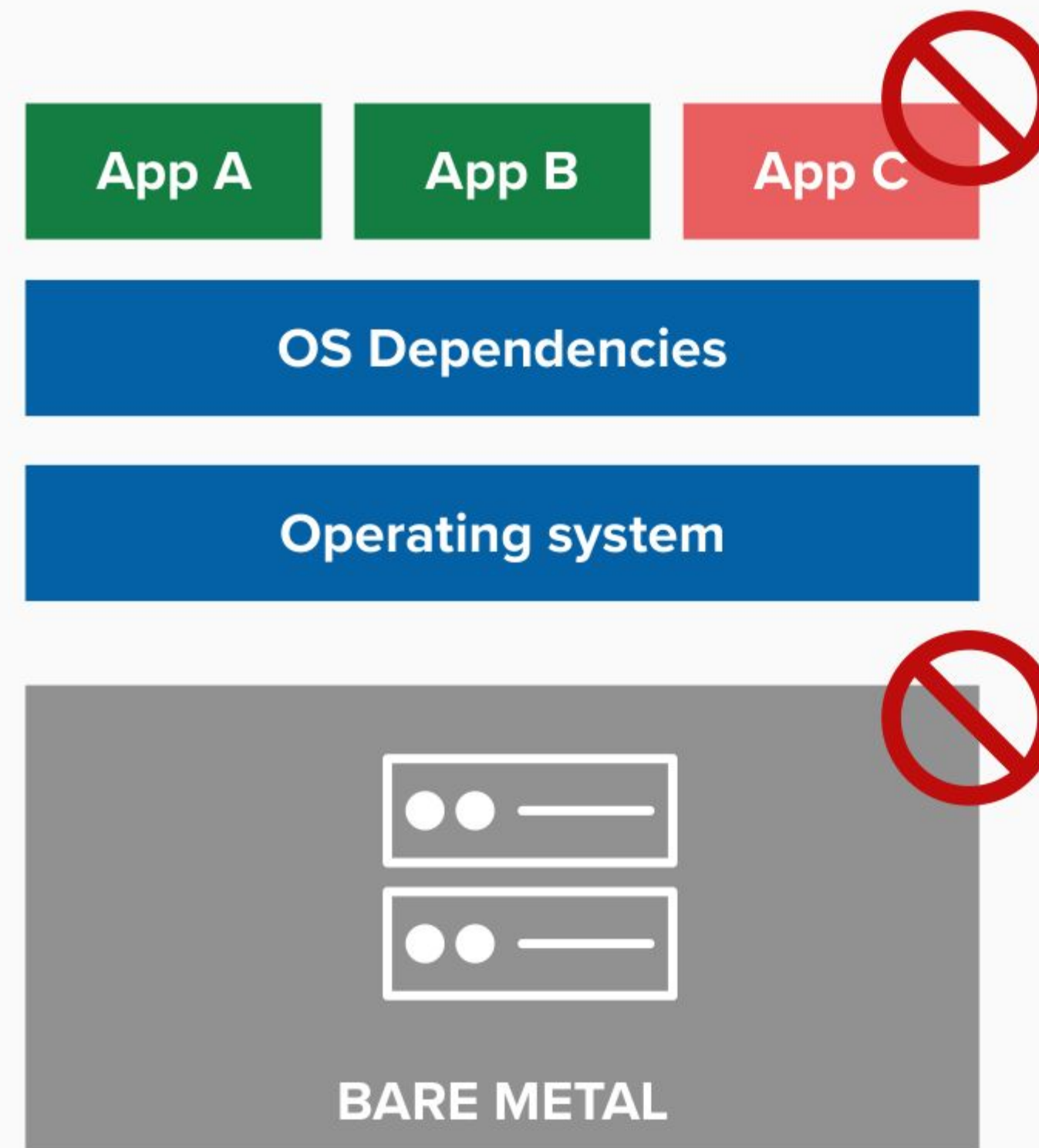
1. Traditional deployment



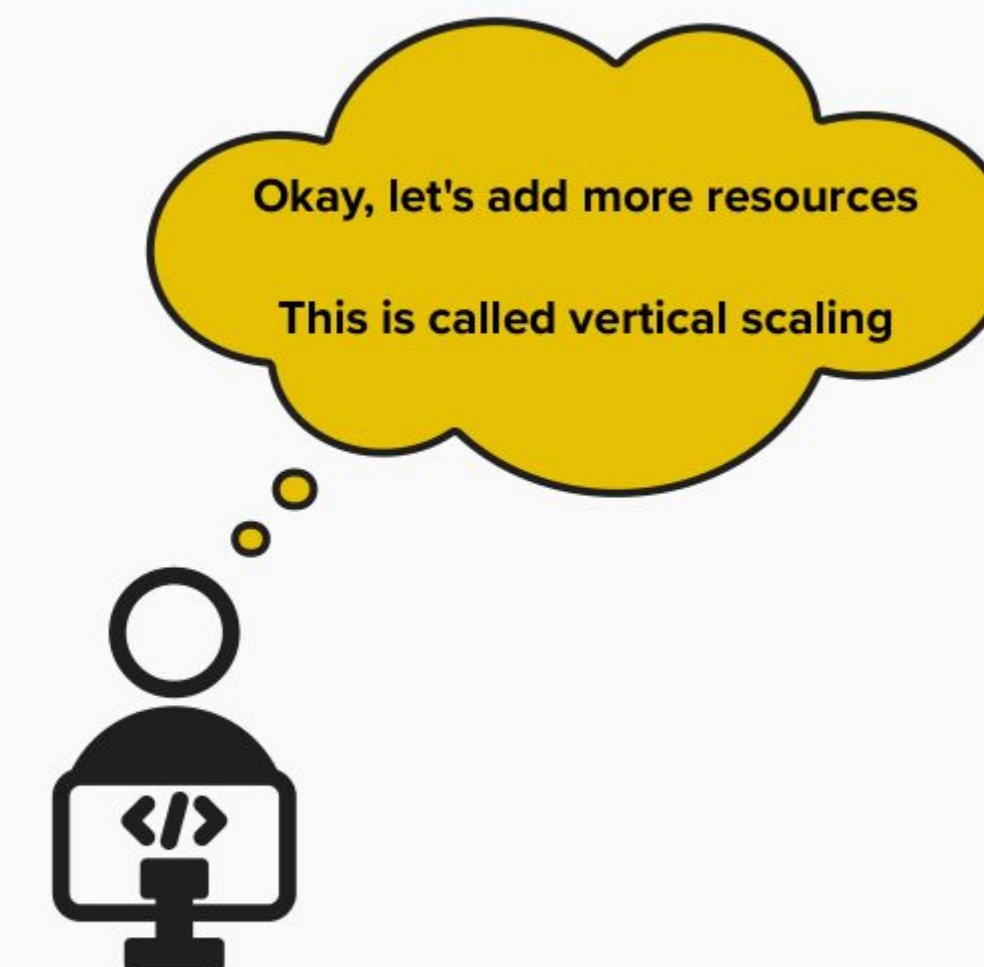
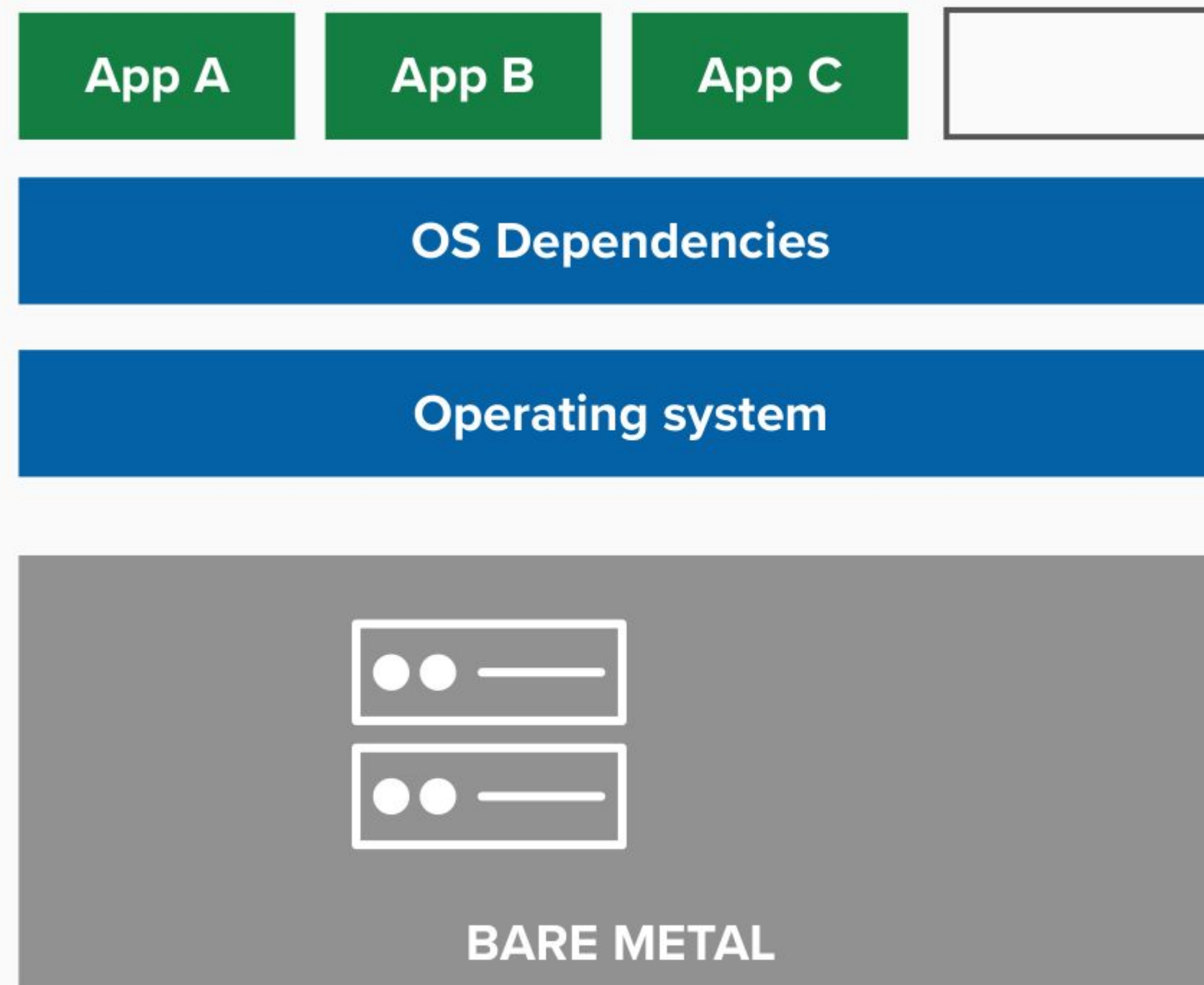
1. Dependency Management Challenges



1. Scalability Challenges



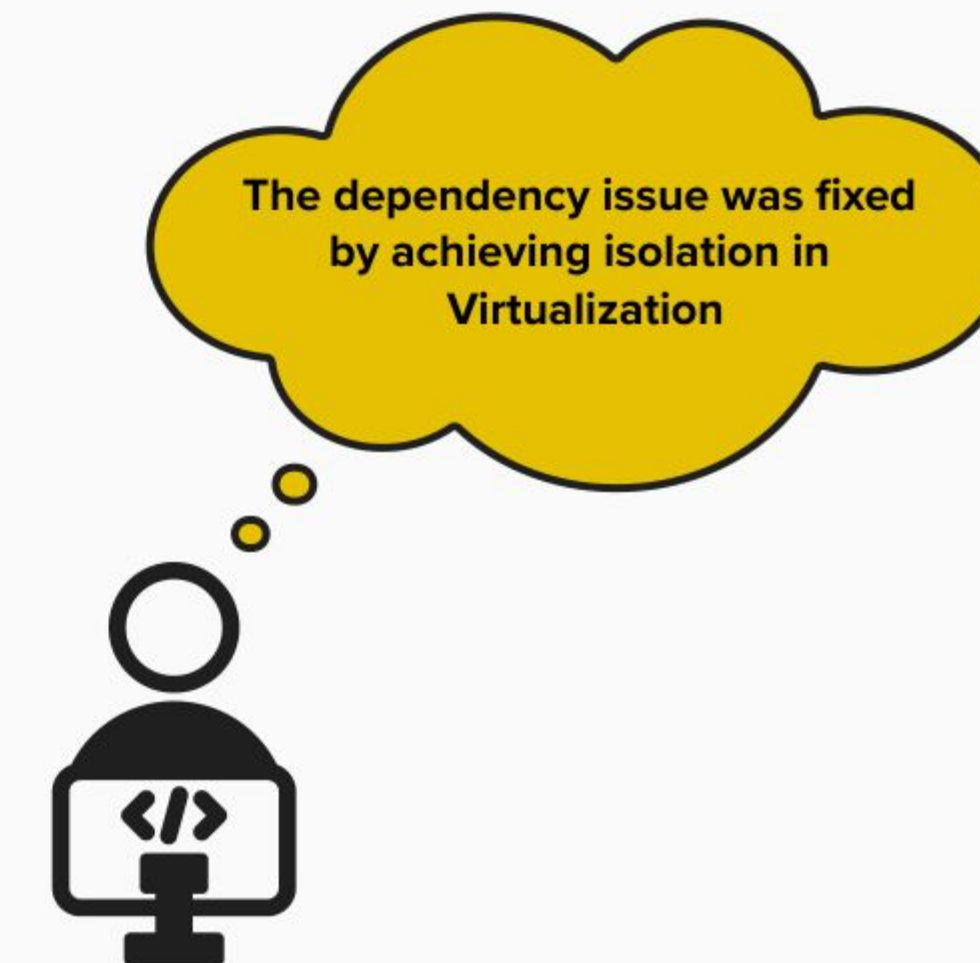
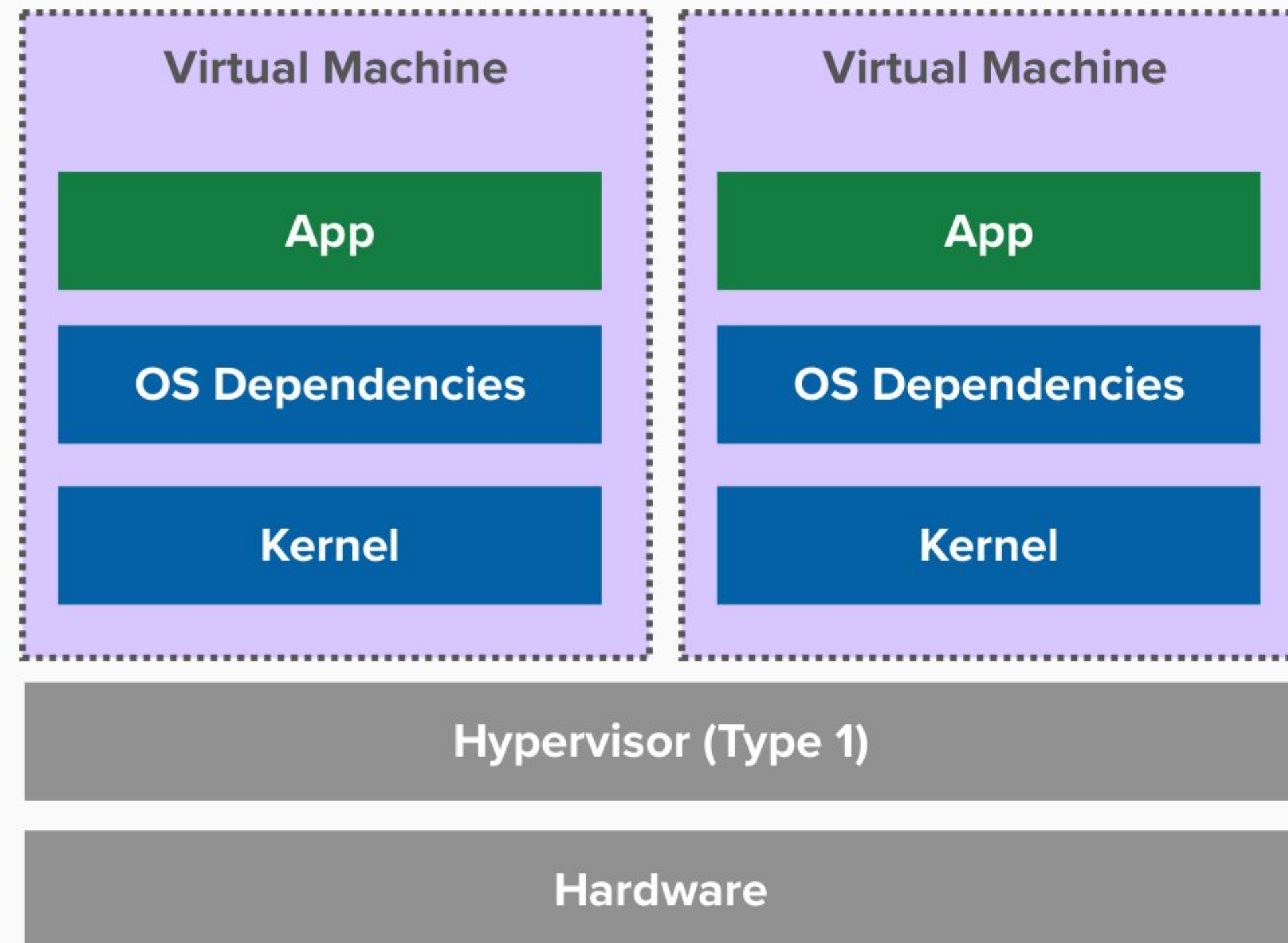
1. Vertical scaling



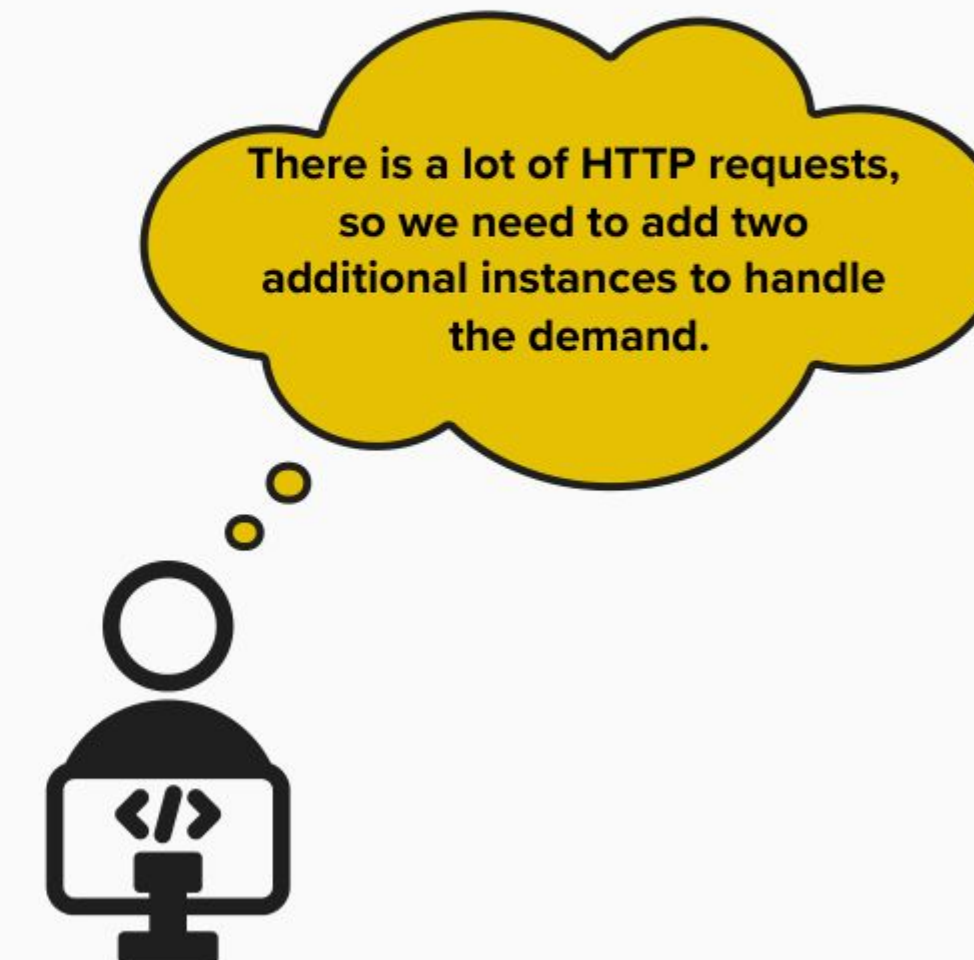
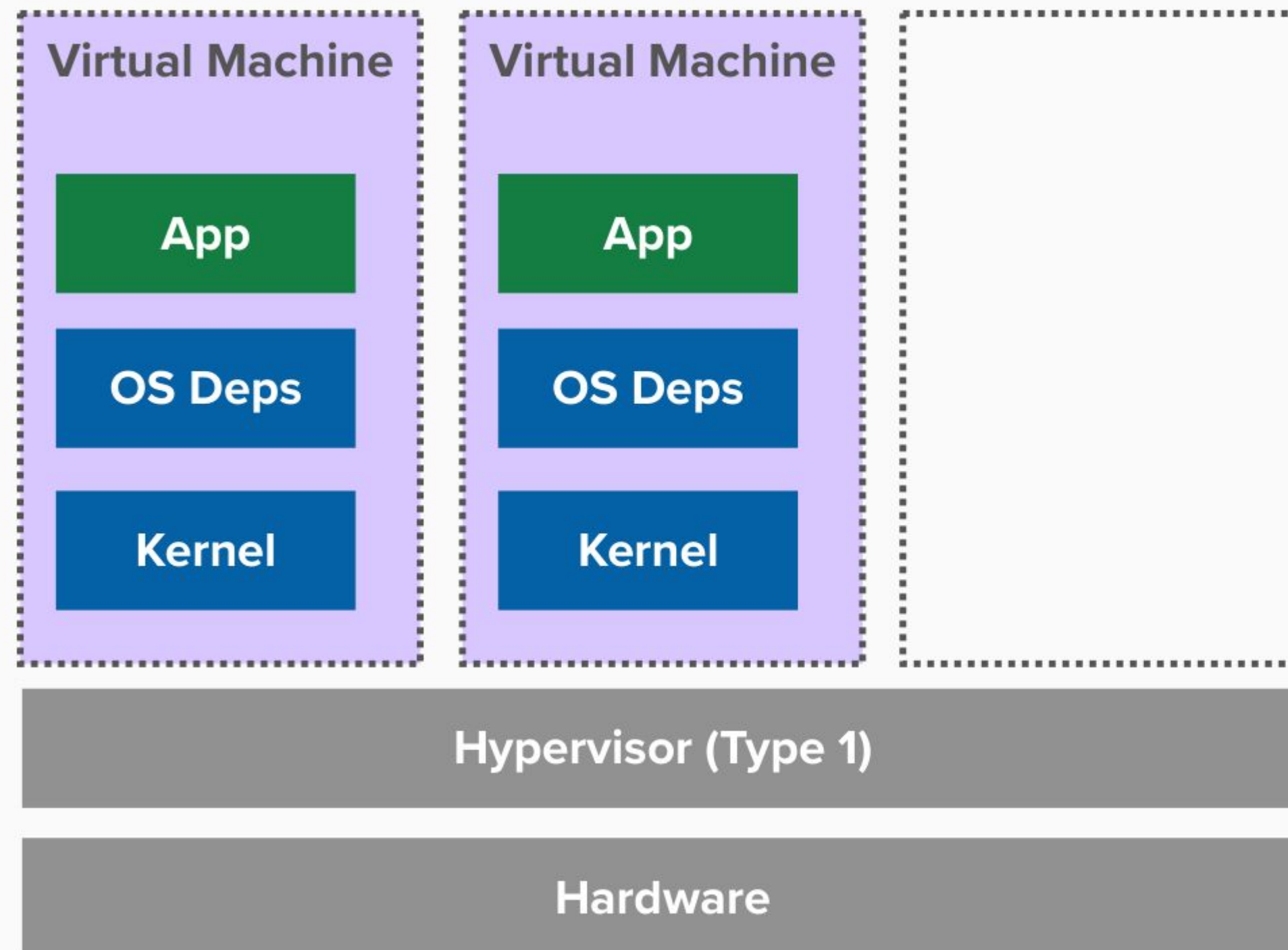
1. Vertical scaling problem

Once we run out of resources again, we will need to keep adding more and more resources to the host. This approach is neither scalable nor maintainable.

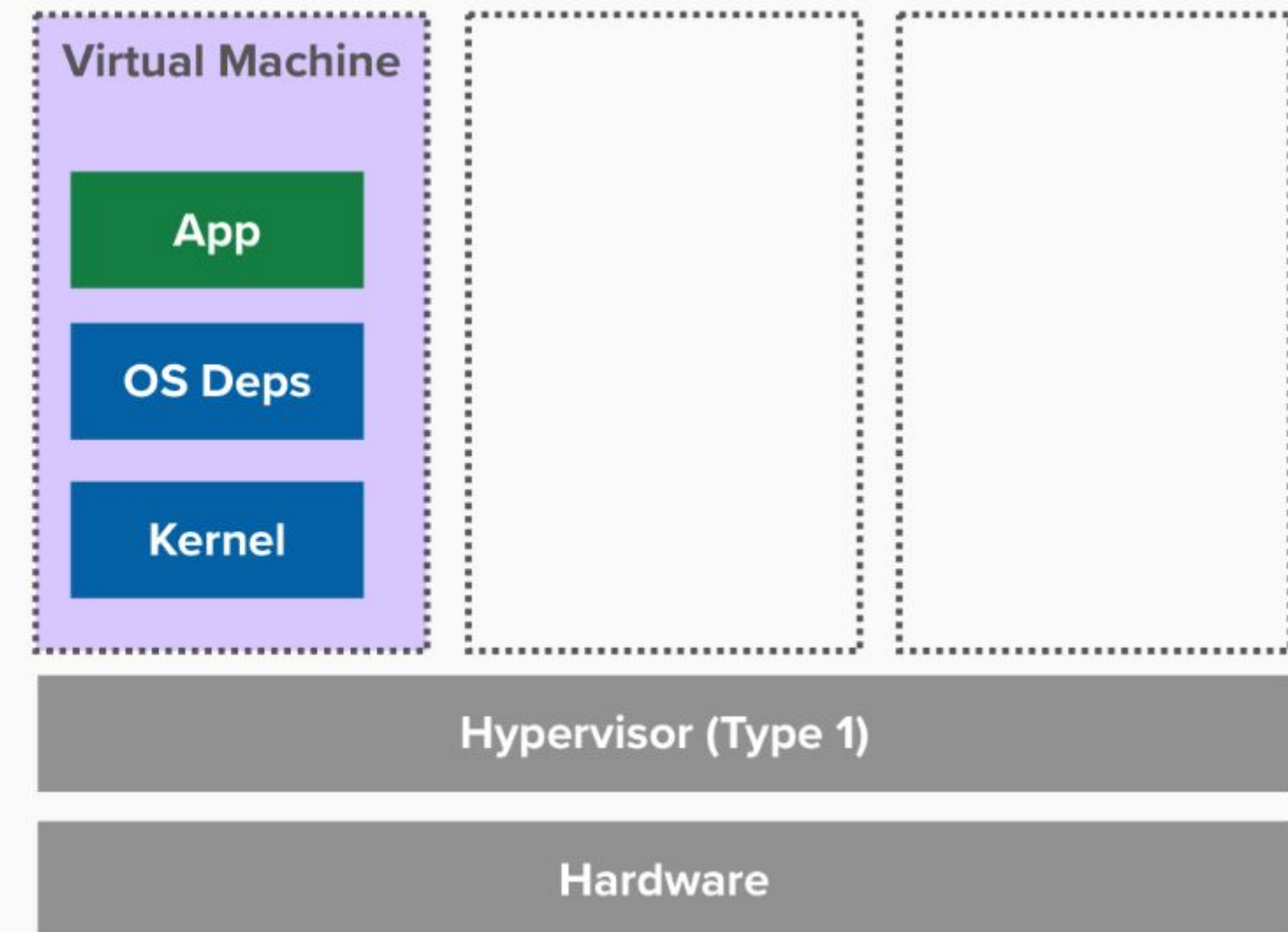
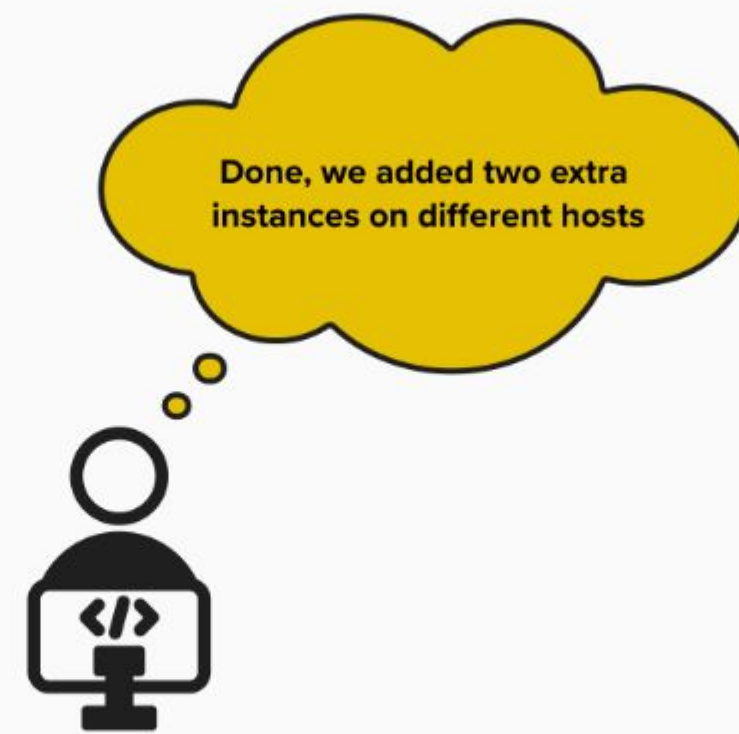
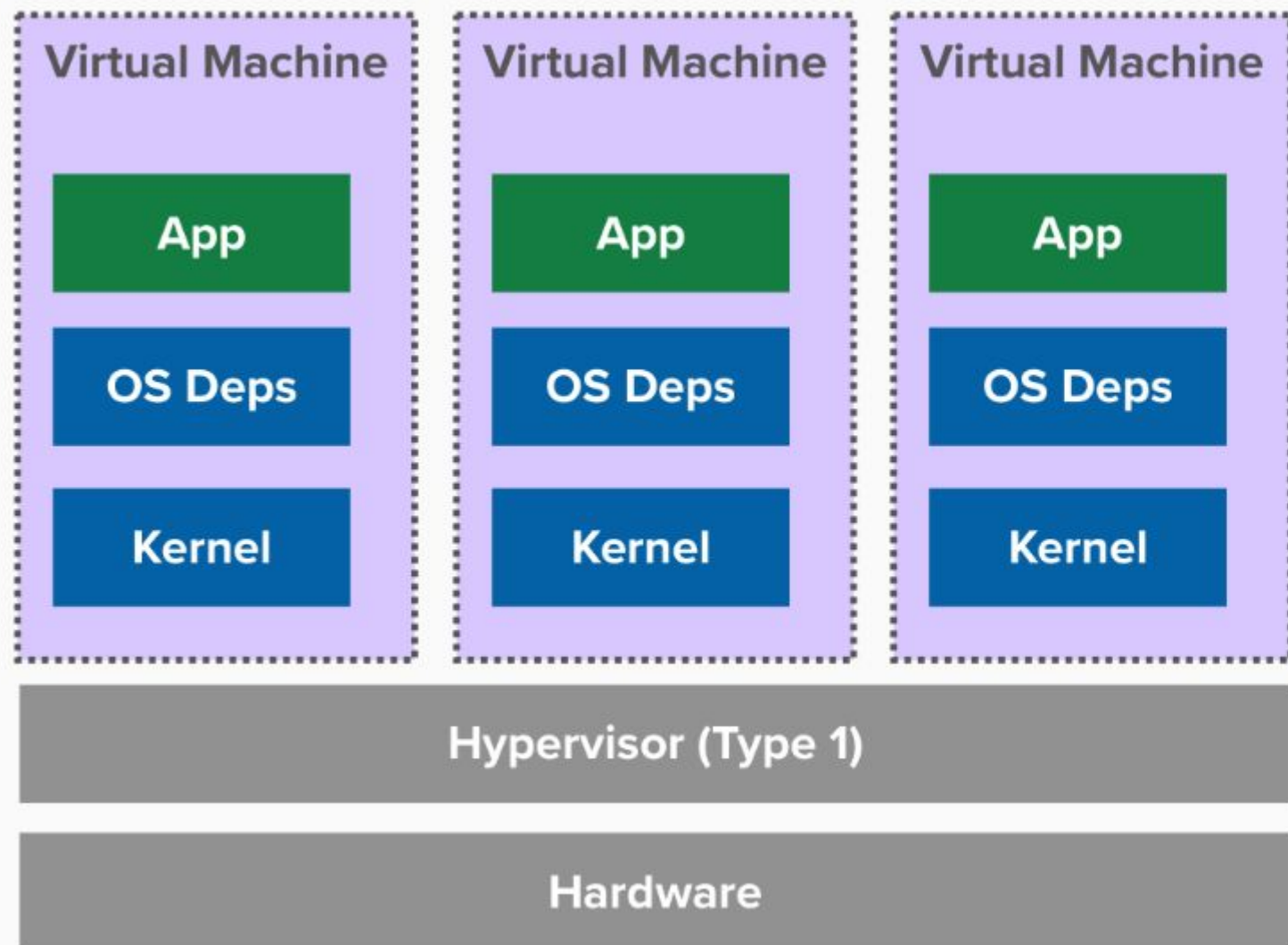
1. Virtualization as a Solution for Dependency Management Issues



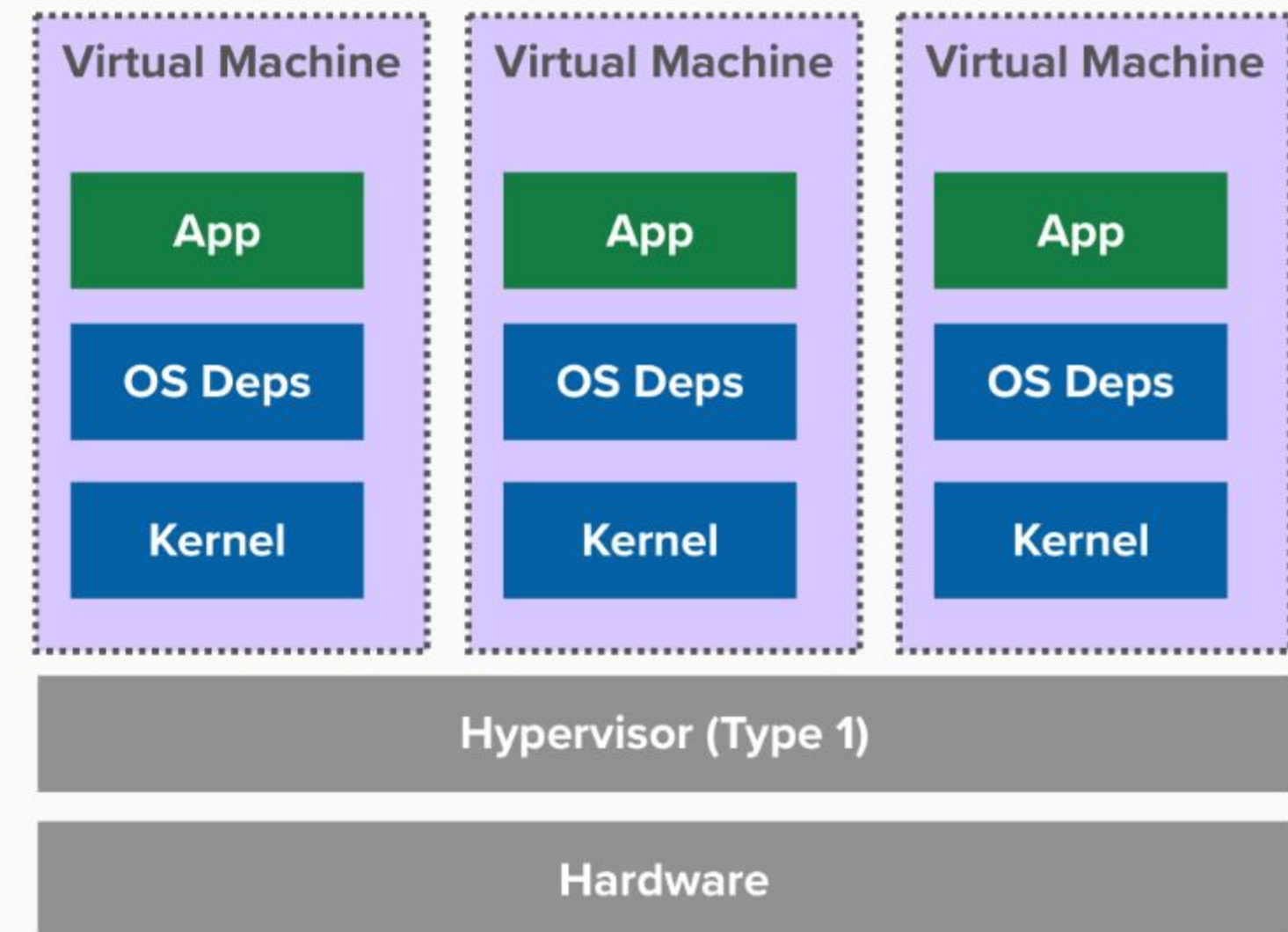
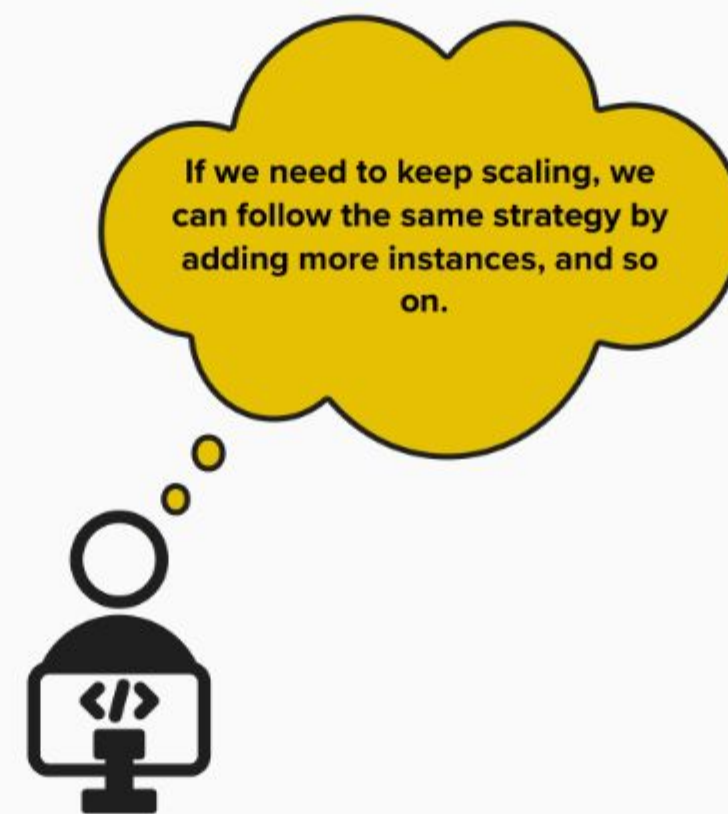
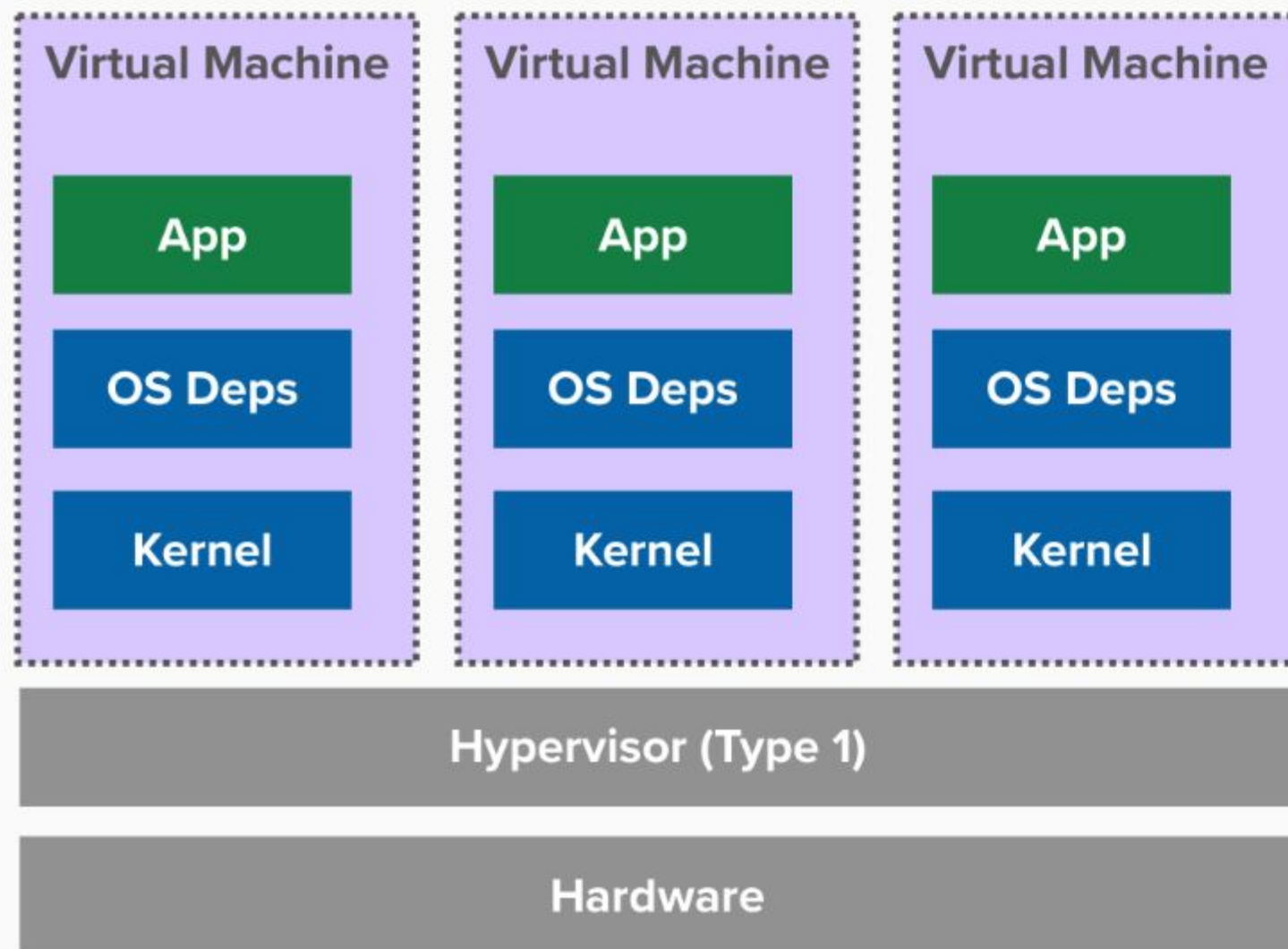
1. Virtualization as a Solution for scalability



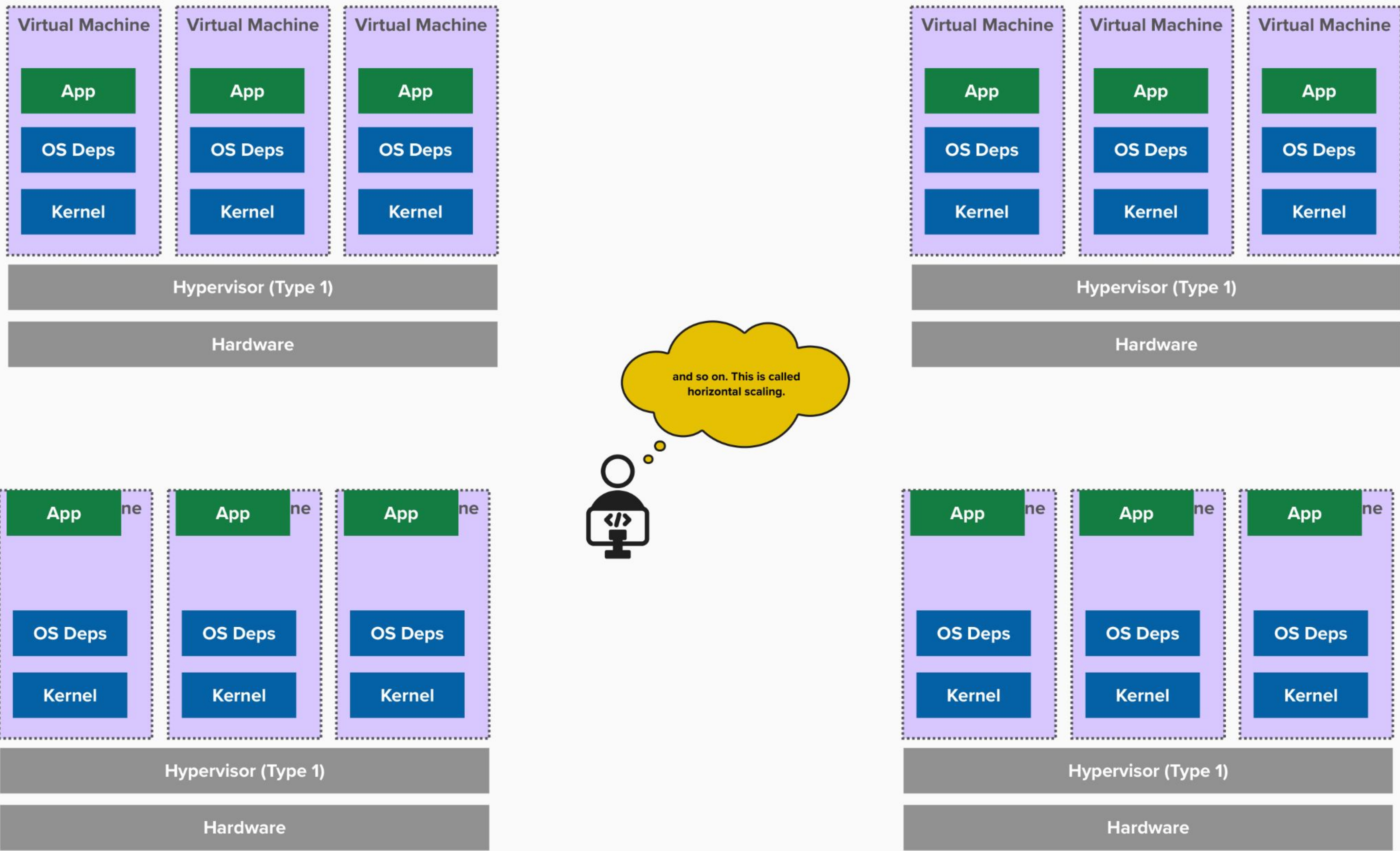
1. Virtualization as a Solution for scalability



1. Virtualization as a Solution for scalability



1. Virtualization as a Solution for scalability

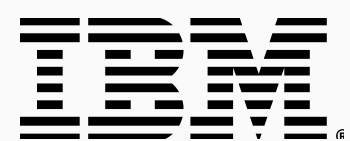


1. Virtualization as a Solution for scalability

However, there are some challenges.

How can we spin up VMs faster?

How can we ensure portability when VMs are migrated between hosts?



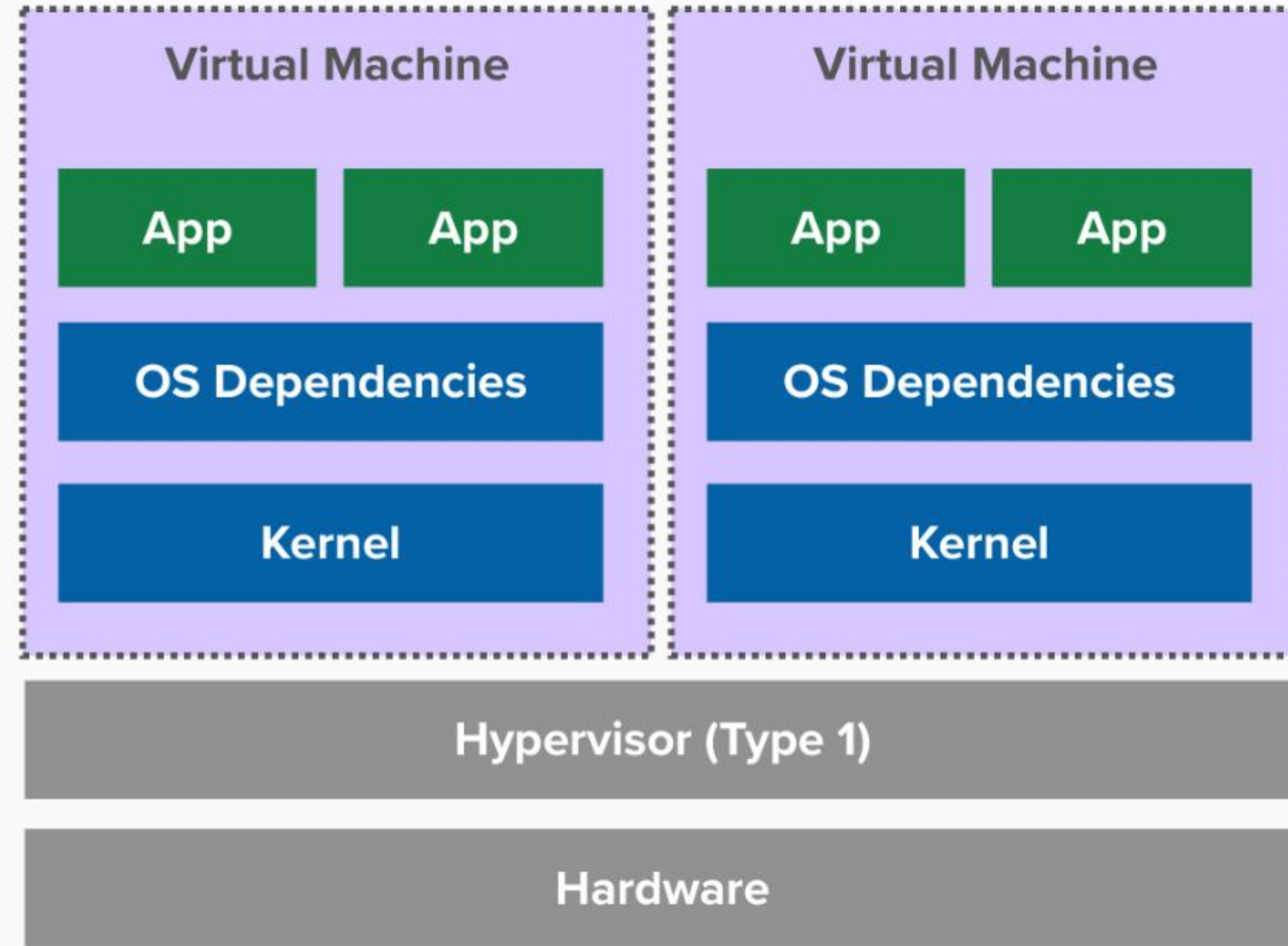
CONTAINERIZATION AS A SOLUTION



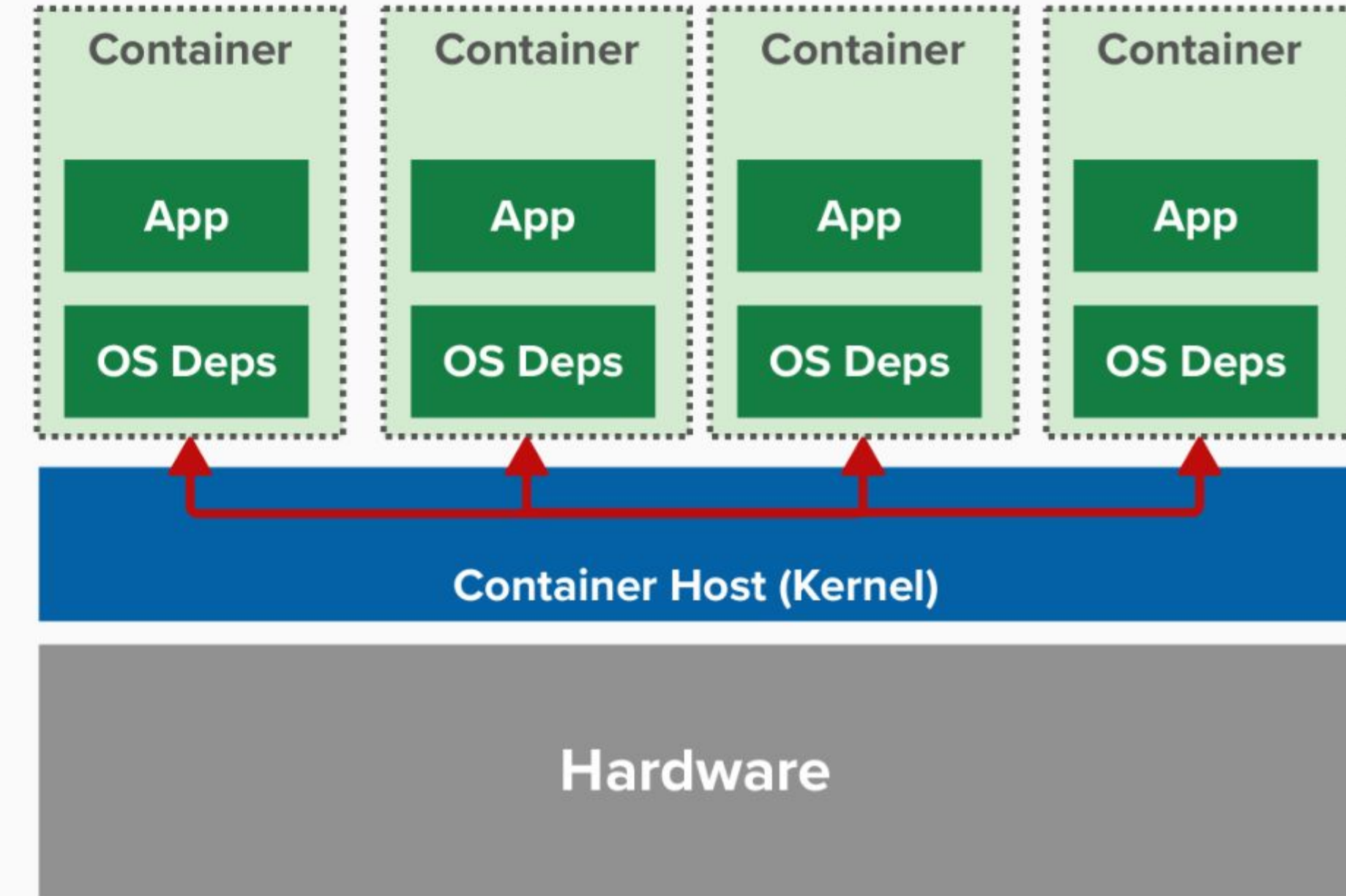
Red Hat



1. Virtual Machines and Containers

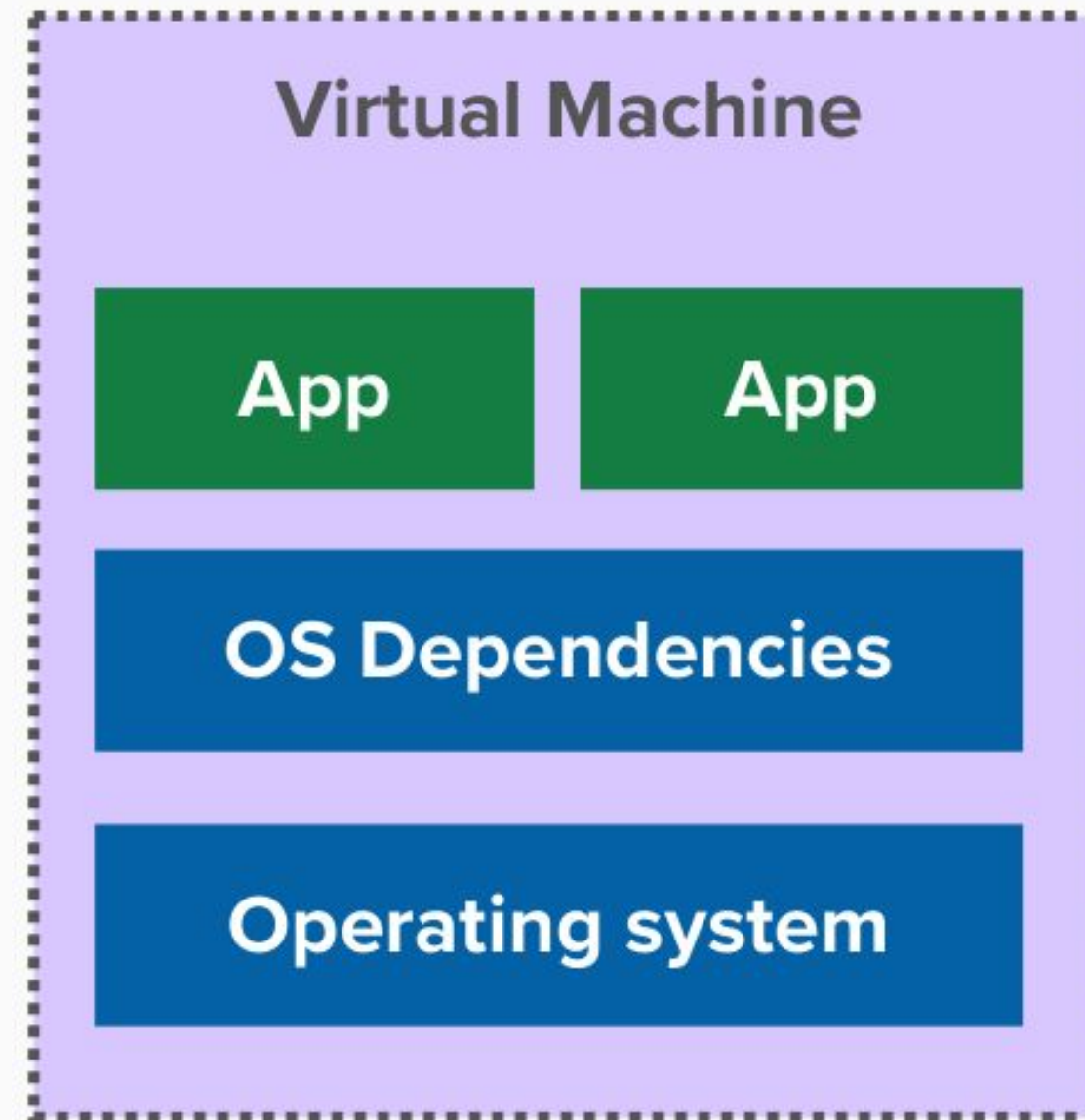


- Virtual machines are isolated
- App are not

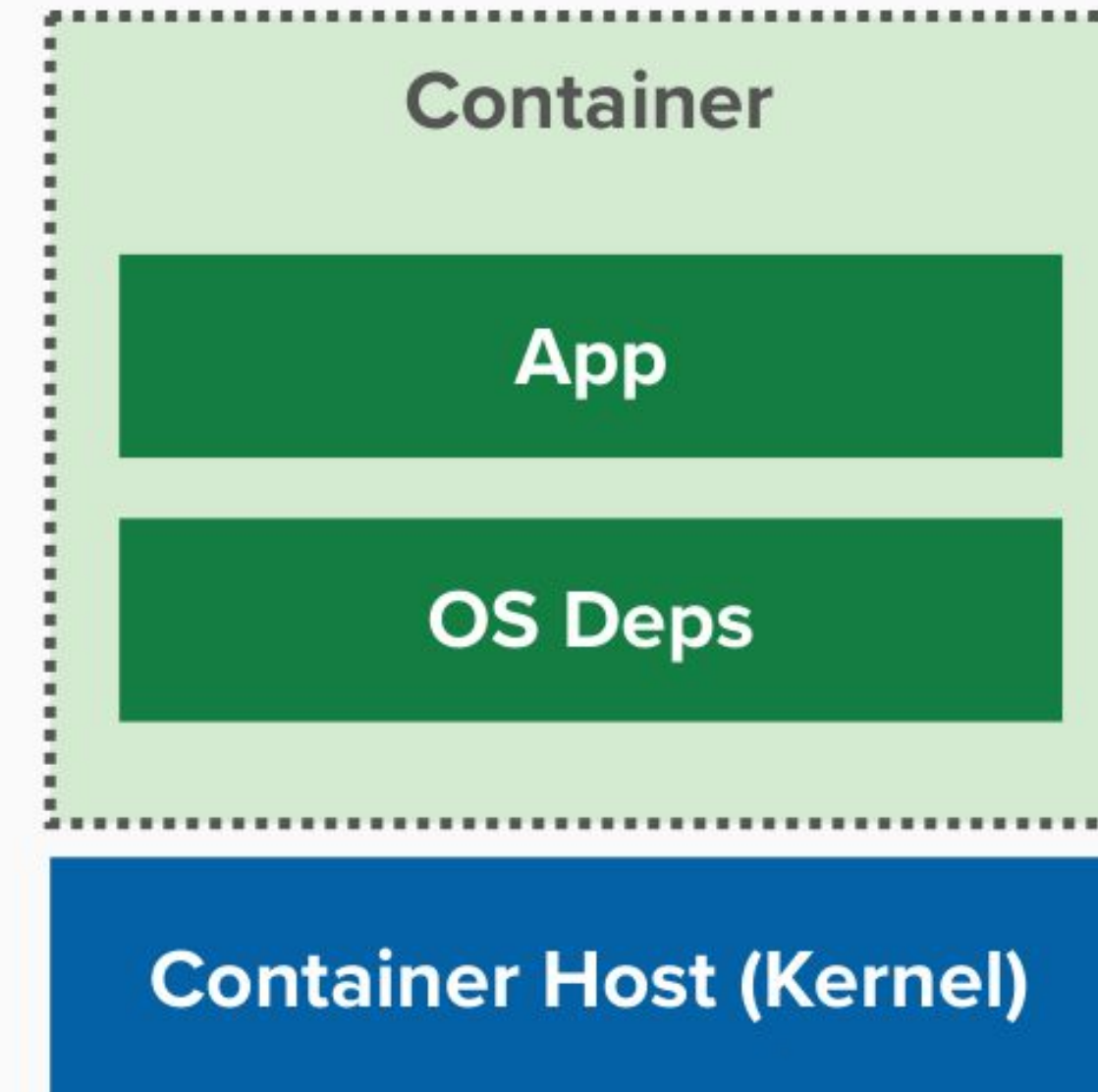


- Containers are isolated
- App are as well

1. Virtual Machines and Containers

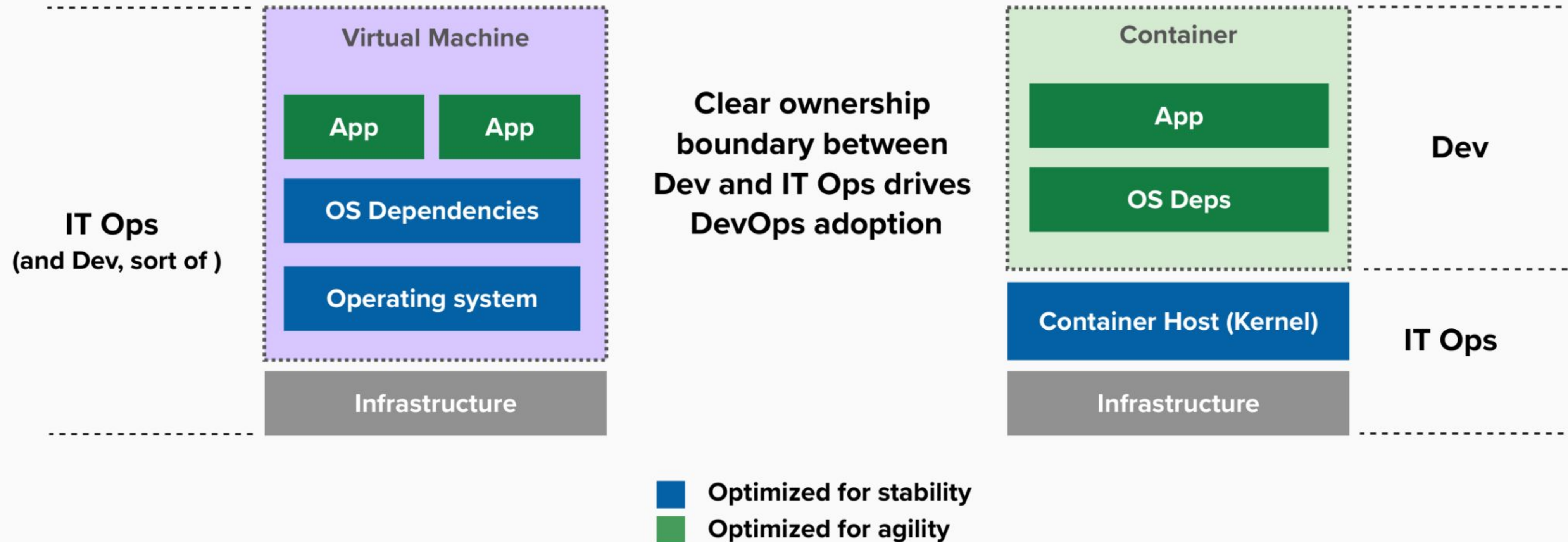


- + VM Isolation
- Complete OS
- Static Compute
- Static Memory
- High Resource Usage



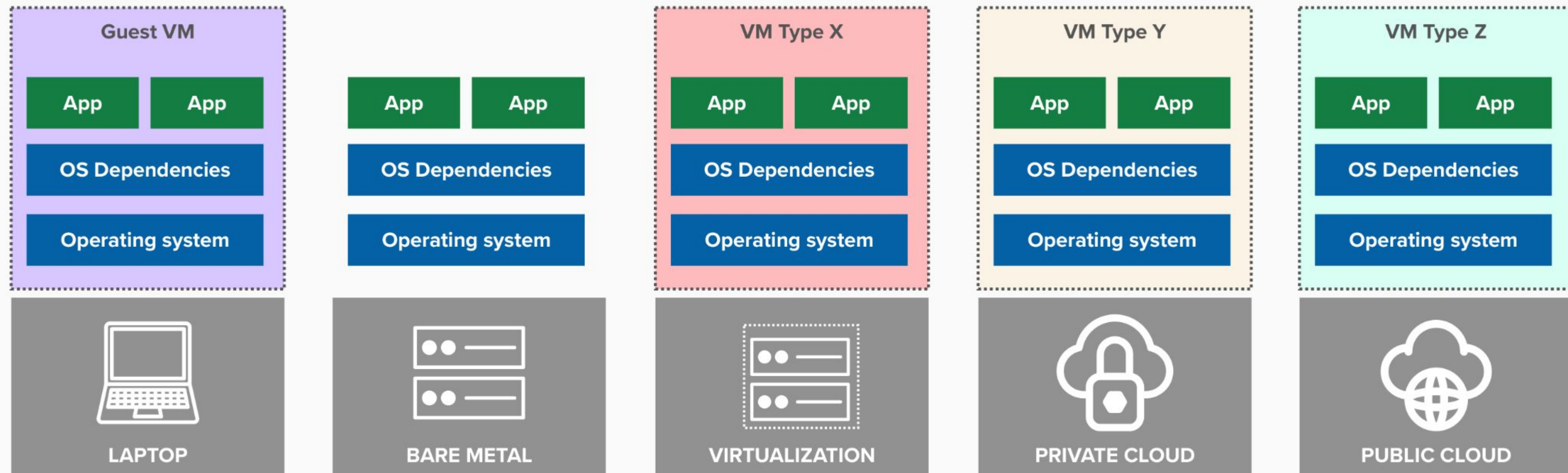
- + Container Isolation
- + Shared kernel
- + Burstable Compute
- + Burstable Memory
- + Low Resource Usage

1. Virtual Machines and Containers



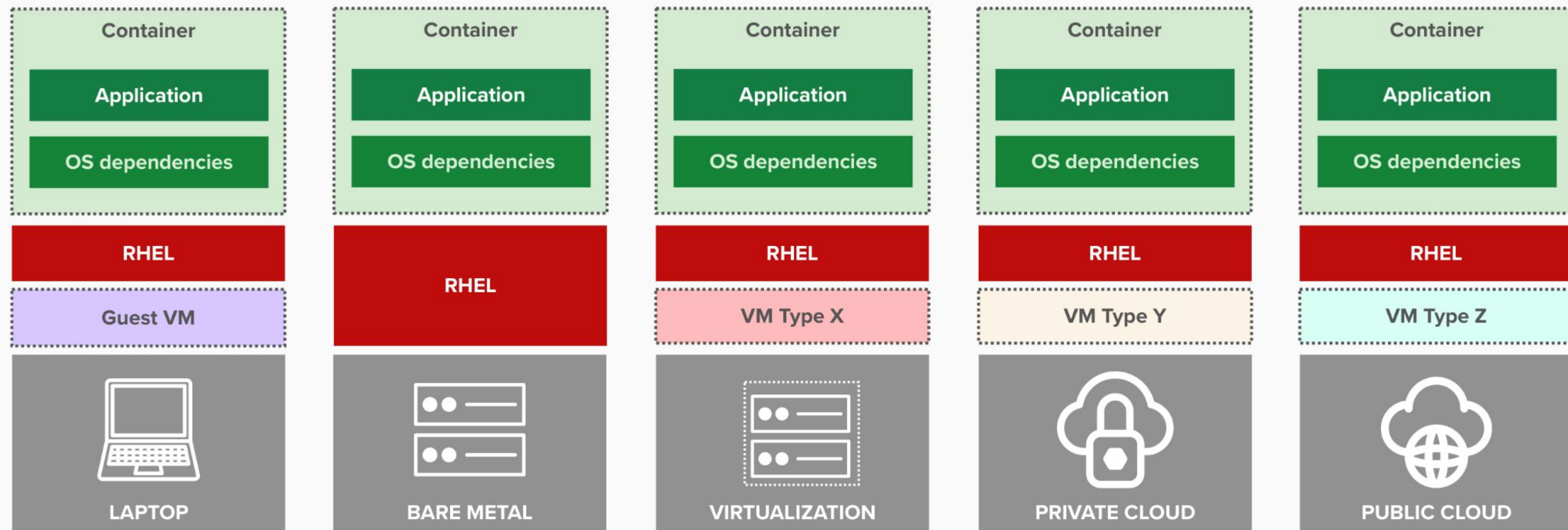
1. Application Portability with VM

Virtual Machines are **NOT** portable across hypervisors and do **NOT** provide portable packaging for applications

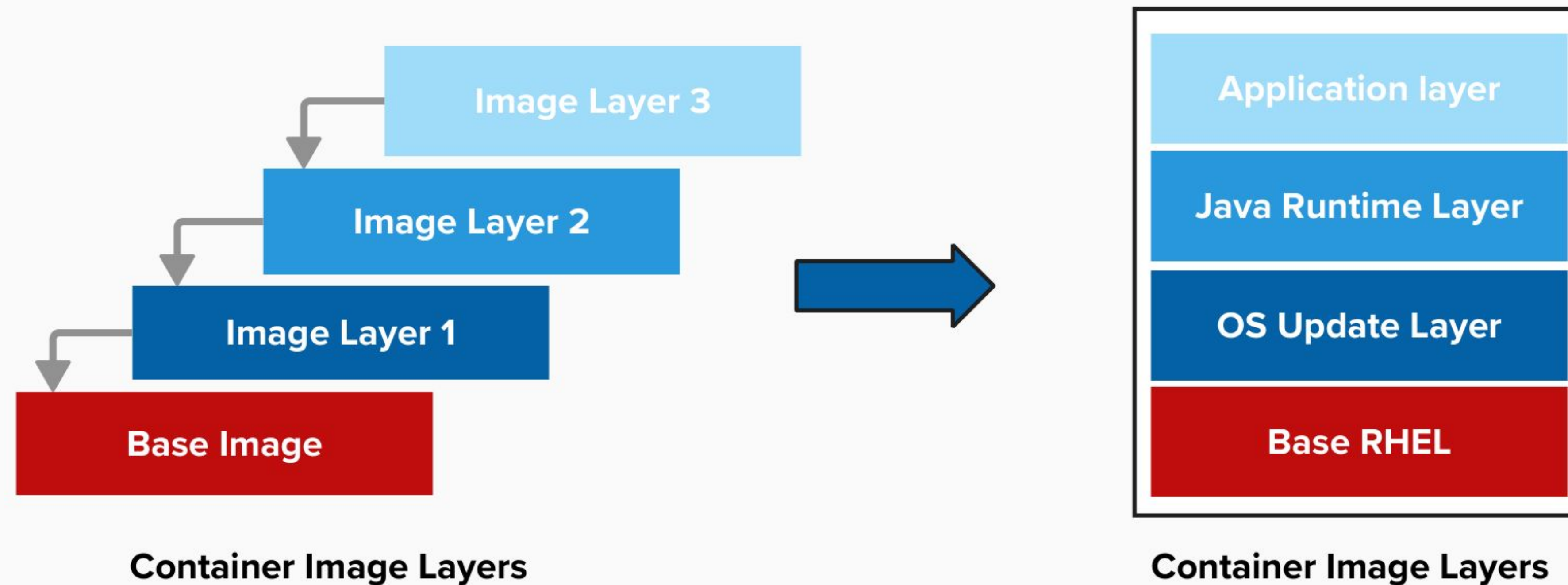


1. Application Portability with Containers

RHEL Containers + RHEL Host = Guaranteed Portability Across Any Infrastructure



1. Rapid Security Patching Using Container Image Layering



CONTAINER REGISTRY

TEXT

```
apiVersion: batch/v1beta1
kind: CronJob
metadata:
  name: cronjob-example
spec:
  schedule: "*/1 * * * *"
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      completions: 4
      parallelism: 2
      template:
        spec:
          containers:
            - name: hello
              image: alpine:latest
              command: ["/bin/sh", "-c"]
              args: ["echo hello from $HOSTNAME!"]
              restartPolicy: Never
```

```
$ kubectl get jobs
```

NAME	DESIRED	SUCCESSFUL	AGE
cronjob-example-1519053240	4	4	2m
cronjob-example-1519053300	4	4	1m
cronjob-example-1519053360	4	4	26s

```
$ kubectl describe cronjob cronjob-example
Name: cronjob-example
Namespace: default
Labels: <none>
Annotations: <none>
Schedule: */1 * * * *
Concurrency Policy: Allow
Suspend: False
Starting Deadline Seconds: <unset>
Selector: <unset>
Parallelism: 2
Completions: 4
Pod Template:
  Labels: <none>
  Containers:
    hello:
      Image: alpine:latest
      Port: <none>
      Command:
        /bin/sh
        -c
      Args:
        echo hello from $HOSTNAME!
      Environment: <none>
      Mounts: <none>
      Volumes: <none>
Last Schedule Time: Mon, 19 Feb 2018 09:54:00 -0500
Active Jobs: cronjob-example-1519052040
Events:
  Type    Reason             Age   From                    Message
  ----    -
  Normal  SuccessfulCreate   3m    cronjob-controller      Created job cronjob-example-1519051860
  Normal  SawCompletedJob    2m    cronjob-controller      Saw completed job: cronjob-example-1519051860
  Normal  SuccessfulCreate   2m    cronjob-controller      Created job cronjob-example-1519051920
  Normal  SawCompletedJob    1m    cronjob-controller      Saw completed job: cronjob-example-1519051920
  Normal  SuccessfulCreate   1m    cronjob-controller      Created job cronjob-example-1519051980
```



Red Hat

